

Documentación Técnica del Frontend Gestor de Sueldos "25 de Mayo"



**COOPERATIVA DE
SERVICIOS PÚBLICOS**
25 de Mayo
Ltda.

Descripción del Documento

Este documento describe la arquitectura, componentes, páginas, servicios de API, flujos de usuario y estructura interna del frontend del Gestor de Sueldos "25 de Mayo", desarrollado en ReactJS. El objetivo es servir como guía para desarrolladores actuales y futuros, así como documentación formal para la presentación del proyecto.





Stack Tecnológico del Frontend



ReactJS

Create React App



React Router DOM

Para navegación



Axios

Para comunicación con el backend



SCSS

Para estilos



Framer Motion

Para animaciones



Lucide React Icons

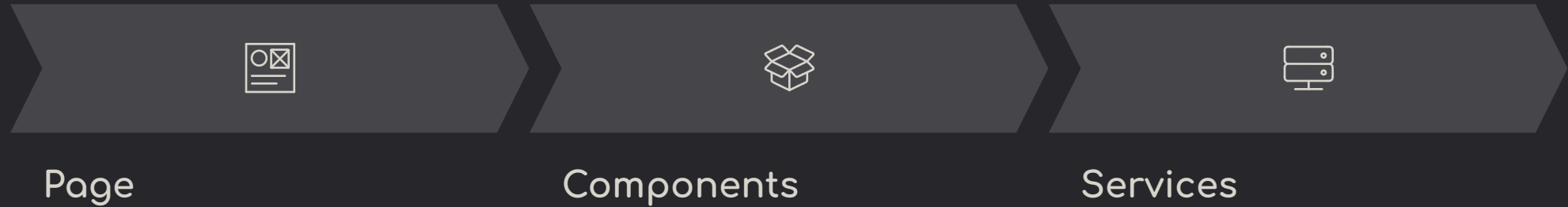
Para iconografía

html2pdf.js para generación de PDF en pantalla de liquidaciones

Arquitectura Interna del Frontend

Patrón de organización

El frontend está construido bajo un patrón



Esto permite separar claramente:

- Lógica de negocio frontend
- Componentes visuales
- Comunicación con el backend

Pages: Vistas Completas de la Aplicación

Representan vistas completas de la aplicación, cada una con su propia lógica.

Dashboard

Pantalla principal, resume accesos y se diseñó como punto de inicio de la navegación.

Employees

Funciones:

- Listado de empleados con tabla
- Búsqueda y filtros
- Botón para ver detalle
- Botón de edición / actualización

EmployeeDetail

Permite:

- Ver información detallada del empleado
- Ver categorías, zona, convenio ligado
- Ver historial de liquidaciones (cuando aplica)

Liquidacion

Pantalla clave que incluye:

- Selección de empleado
- Carga de conceptos variables
- Vista previa de cálculo
- Envío del formulario al backend
- Render de la liquidación generada
- Descarga de PDF con html2pdf

Pages: Convenios e Historial

Convenios

Lista de todos los convenios

- Nombre
- Categorías asociadas
- Botón para ver detalle

ConvenioDetail

Visualiza

- Categorías del convenio
- Zonas
- Bonificaciones fijas
- Bonificaciones variables
- Descuentos laborales

HistorialPagos

Listado de todas las liquidaciones registradas que contiene:

- Tabla de pagos
- Filtros por empleado o período
- Acceso al detalle por cada pago

Components

Reutilizables para toda la aplicación:



Sidebar

- Menú lateral fijo
- Navegación a todas las vistas
- Iconografía con Lucide



Header

- Título de la vista
- Botón de acciones secundarias



Modal

Reutilizable para formularios o confirmaciones.



Form Inputs

- Select
- Input básico
- Input numérico
- TextArea



Tooltip

Para mostrar ayuda contextual.

Servicios (API Layer)

Toda comunicación con backend está centralizada en `src/services/`.

1

axiosClient.js

Define el cliente Axios base,
Incluye interceptores para
autenticación JWT.

2

empleadosAPI.js

Funciones expuestas:

- getEmpleados()
- getEmpleado(id)
- createEmpleado(data)
- updateEmpleado(id, data)
- deleteEmpleado(id)

3

conveniosAPI.js

Proporciona:

- getConvenios()
- getConvenioById(id)
- getCategoriasByConvenio(id)

4

liquidacionesAPI.js

Incluye:

- liquidarSueldo(data) → POST /liquidaciones
- getPagos()
- getPagoDetalle(id)
- getPagosPorEmpleado(idEmpleado)

5

categoriasAPI.js

Permite:

- Obtener categoría por ID
- Crear categoría
- Listar categorías por convenio



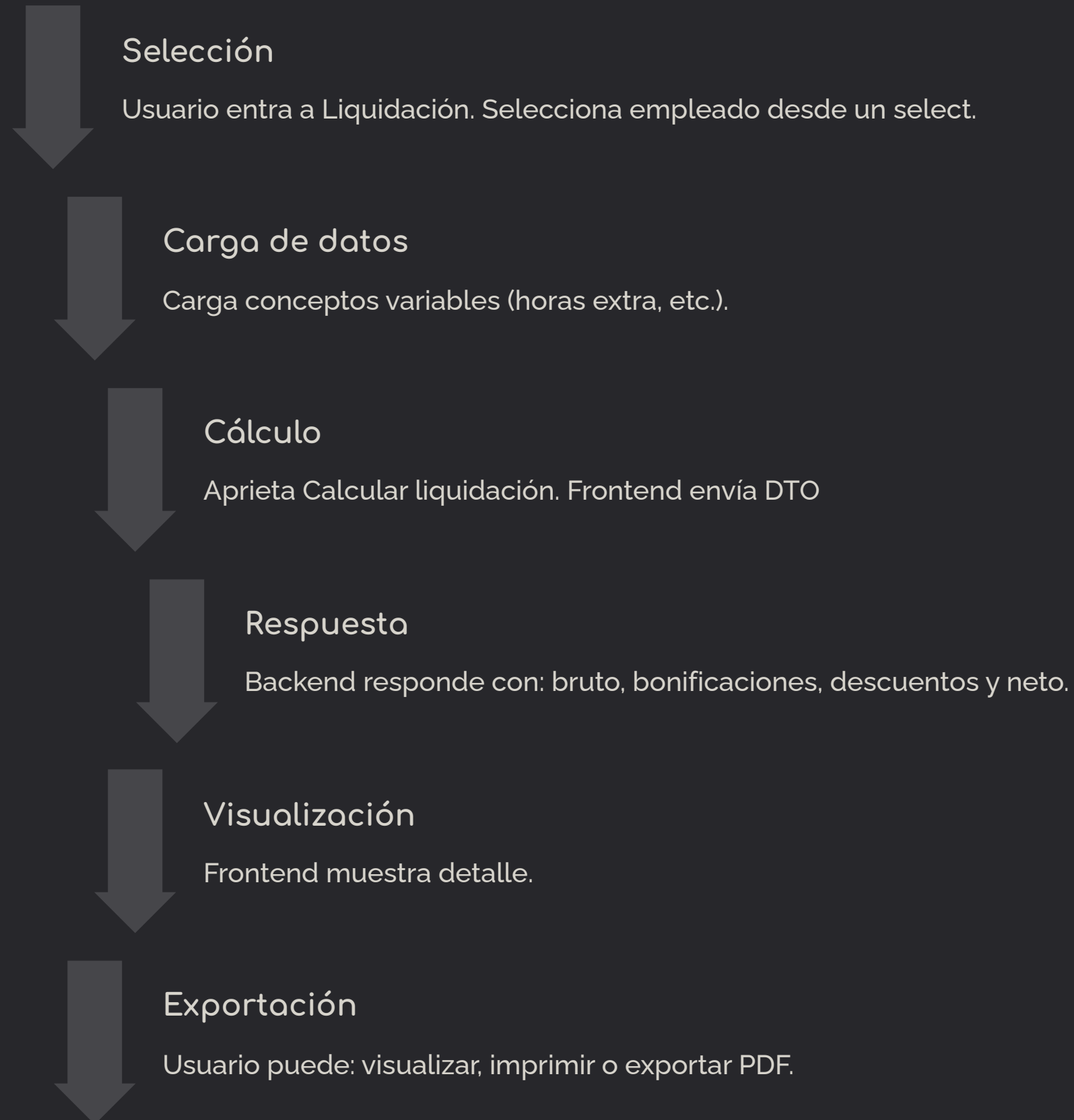
Flujos de Usuario

Cada vista tiene un flujo concreto conectado al backend.

Flujo: Alta y gestión de empleados

- 1 — Usuario va a página Empleados.
- 2 — React carga empleados vía `getEmpleados()`.
- 3 — Usuario toca "Agregar empleado".
- 4 — Abre modal con formulario.
- 5 — Al confirmar: → `POST /api/empleados`
- 6 — UI recarga lista.

Flujo: Liquidación de sueldos



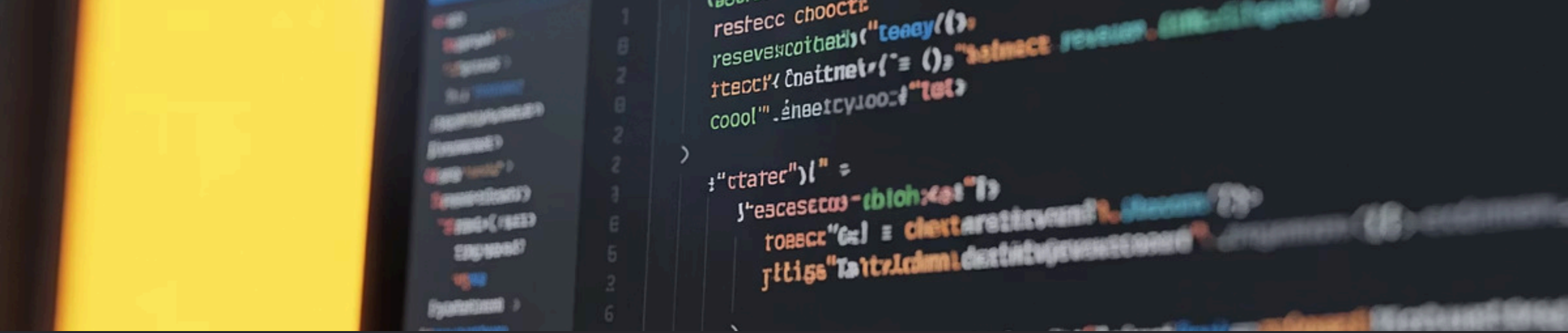
Flujos: Consulta de pagos y Gestión de Convenios

Flujo: Consulta de pagos

1. Usuario hace click en Historial de Pagos.
2. Se llama: GET /api/pagos.
3. Tabla muestra: empleado, período, bruto, neto
4. Al seleccionar: → GET /api/pagos/{id} Carga vista con el desglose completo.

Flujo: Gestión de Convenios

1. Página Convenios llama: getConvenios().
2. Usuario entra a un convenio → /convenios/:id.
3. Se carga toda la información: categorías, zonas, bonificaciones, descuentos.



Manejo de Estado y Ciclo de Vida

El frontend utiliza React Hooks:

useState

Para almacenar datos.

useEffect

Para cargar datos desde la API.

useCallback

Para memorizar funciones.

useNavigate

Para mover entre rutas.

📌 **No se utiliza Redux ni context global**, manteniendo la app simple y modular.

Estilos (SCSS)

La app utiliza estilos personalizados bajo

- src/styles/
- src/Components/_/_scss
- src/Pages/_/_scss

Estilo característico:

Colores suaves

Sombras y bordes redondeados

Diseño limpio y profesional



Generación de PDF

En la vista de liquidación se usa **html2pdf.js** para convertir un bloque HTML en un recibo descargable.

Funcionalidad:

- Captura el detalle de la liquidación
 - Exporta a PDF con formato coherente
 - Ideal para entregar a empleados



Manejo de Errores

El frontend maneja errores mediante:



`console.error()`



`alert()` o modales de error
(según vista)



Validaciones en formularios



Control de estado de red
(`try/catch` en APIs)

